You said:

I am planning to make a mod for quake.
It should at least run on the rerelease and on ironwail.
It will have new enemies and weapons.
I know how to craft assets.
I downloaded the source from here:
https://github.com/id-Software/quake-rerelease-qc
All i want for now is how to setup en enviroment where i can compile and run this. For now i dont want to know anything more.
I am on pop!os by the way so if there are native linux versionms of tools i would prefer them, but using wine is also ok.

Worked for 14s

ChatGPT said:

For your setup, I would aim for a **native Linux development environment on Pop!_OS**, with Wine only if a particular tool turns out to require Windows.

There are two things to keep separate:

1. **Compiling the QuakeC source** into game code.
2. **Running and testing the resulting mod** in the Quake rerelease and Ironwail.

You shouldn't need a Windows development environment for either of these.

I'll keep this strictly to environment setup, installation, compilation and launching. No weapon/enemy implementation or mod-design discussion.

1. Install the QuakeC compiler

The official repository explicitly recommends **FTEQCC**. It supports the advanced QuakeC features used by the rerelease source. id-Software/... +1

Good news: FTEQCC has a native 64-bit Linux command-line version. No Wine required.

Download it here:

- FTEQCC official website ↗
- Choose **Linux x86_64**.

- Extract the archive somewhere convenient, for example `~/tools/fteqcc/`.

Make the compiler executable:

```
chmod +x ~/tools/fteqcc/fteqcc64
```

You can test it with:

```
~/tools/fteqcc/fteqcc64
```

You should see its command-line help.

You can also install the packaged version through APT:

```
sudo apt update
sudo apt install fteqcc
```

However, I recommend the official binary to avoid potential differences between compiler versions.

2. Prepare your project directory

I recommend keeping your development files separate from your original game installation.

For example:

```
~/quake-dev/
├── source/
│   └── quake-rerelease-qc/
│       ├── quakec/
│       ├── quakec_ctf/
│       ├── quakec_hipnotic/
│       ├── quakec_mg1/
│       ├── quakec_mg3/
│       └── quakec_rogue/
│
└── builds/
```

```
    └── mymod/
        └── progs.dat
```

Clone the repository:

```
mkdir -p ~/quake-dev/source
cd ~/quake-dev/source

git clone https://github.com/id-Software/quake-rerelease-qc.git
```

The repository contains six separate QuakeC codebases. For now, start with `quakec`, which is the base Quake campaign.

3. Compile the source

One important detail: the repository's `quakec/progs.src` specifies the output as:

```
../progs.dat
```

This means the compiler places the resulting `progs.dat` in the parent directory of `quakec`.

GitHub

Compile it with:

```
cd ~/quake-dev/source/quake-rerelease-qc/quakec

~/tools/fteqcc/fteqcc64
```

FTEQCC automatically looks for `progs.src` in its current working directory.

After compilation, you should have:

```
quake-rerelease-qc/
├── progs.dat
└── quakec/
    ├── progs.src
    ├── defs.qc
```

```
    ├── weapons.qc
    ├── monsters.qc
    └── ...
```

If compilation succeeds, you've got your first compiled QuakeC binary.

You can also use the compiler's `-src` argument when invoking it from another directory.

4. Set up a test mod directory

Find your Quake installation. For a typical Steam installation on Pop!_OS, it will be somewhere under:

```
~/.steam/steam/steamapps/common/Quake/
```

Create a mod directory alongside the game's `id1` directory:

```
mkdir -p "/path/to/Quake/mymod"
```

For your first test, copy the compiled binary:

```
cp ~/quake-dev/source/quake-rerelease-qc/progs.dat \
   "/path/to/Quake/mymod/progs.dat"
```

Your game directory should look something like this:

```
Quake/
├── id1/
├── rerelease/
└── mymod/
    └── progs.dat
```

Keep your source code in `~/quake-dev`; you don't need to put the entire source repository inside the game installation.

5. Install and run Ironwail

Ironwail has native Linux builds.

Download it from:

Ironwail GitHub releases ↗

Download the Linux x86_64 release and extract it somewhere convenient, for example:

```
~/tools/ironwail/
```

The current release notes specify glibc 2.31 or newer, so Pop!_OS should meet that requirement on a reasonably recent installation. GitHub

Run your mod from the Quake installation directory:

```
cd "/path/to/Quake"

~/tools/ironwail/ironwail -game mymod
```

This is also convenient for development: compile, copy the new `progs.dat`, restart or reload the game, and test.

6. Running the mod in the official rerelease

You can keep the same mod directory and test it in the official rerelease as well.

The important distinction is that the rerelease and Ironwail are separate engines. Although Ironwail supports rerelease content, you should not assume that every KEX-specific QuakeC feature in the official source is supported identically by Ironwail.

I'd establish a working baseline in the official rerelease first, then test the same compiled binary in Ironwail.

One correction to the directory layout above: for the official rerelease, the mod should live inside `Quake/rerelease/`, whereas Ironwail can use a mod directory alongside its `id1` game data. The safest approach is to keep separate copies of the mod folder for the two engines.

For the rerelease, the typical layout is:

```
Quake/
└── rerelease/
    ├── id1/
    └── mymod/
        └── progs.dat
```

You can switch to it from the in-game console using:

```
game mymod
```

This is documented in the community's rerelease modding guide ↗. Steam Com... +1

7. Your initial toolchain

Here's what I'd install and use initially:

Tool	Purpose	Platform
FTEQCC	QuakeC compiler	Native Linux
Git	Source control	Native Linux
VS Code or VSCodium	Source editing	Native Linux
Ironwail	Testing	Native Linux
Quake rerelease	Testing	Native Linux Steam version

No Wine necessary.

One compatibility caveat: the official source targets the rerelease. Ironwail's compatibility with rerelease content does not automatically guarantee compatibility with every rerelease-specific QuakeC extension. That will need to be verified with an actual compiled build.

That's all you need to get started. I'd first get the unmodified base `quakec` source compiling successfully before changing anything.

Useful references:

- Official Quake rerelease source ↗
- FTEQCC downloads ↗
- Ironwail downloads ↗
- Rerelease modding guide ↗

You said:

Ok good paradgm. Will follow this thru and ask questions.
first one:
when i set up a source folder. I put the original files there. Then i modify them hence making a mod. What if i work on several mods tho? I guess i will have to make a new copy of the original source everytime. But wouldnt that mean that putting a separate source folder into each wip mod folder along the other assets wuild be way more practical?

ChatGPT said:

Yes. In fact, I think your proposed approach is the better one.

Each mod should be a self-contained project, including its own copy of the QuakeC source, compiled files and custom assets.

That way, you can work on multiple mods independently without worrying about one project's changes affecting another.

I'd organize things like this:

```
~/quake-dev/
│
├── tools/
│   ├── fteqcc/
│   └── ironwail/
│
├── base-source/
│   └── quake-rerelease-qc/
│
└── projects/
    │
    ├── my-first-mod/
```

```
│   ├── source/
│   │   ├── quakec/
│   │   ├── quakec_hipnotic/
│   │   └── ...
│   │
│   ├── assets/
│   ├── build/
│   └── README.md
│
├── my-second-mod/
│   ├── source/
│   ├── assets/
│   ├── build/
│   └── README.md
│
└── my-third-mod/
    ├── source/
    ├── assets/
    ├── build/
    └── README.md
```

The advantage is that you can simply copy the original repository into each project when you start a new mod.

A slightly better approach: use Git

I would recommend making every mod its own Git repository.

For example:

```
cd ~/quake-dev/projects/my-first-mod

git init
git add .
git commit -m "Initial project based on Quake rerelease QC"
```

Now you have a history of all your modifications and can revert changes if something breaks.

The original repository is licensed under GPLv2, so keep its `COPYING.txt` and licensing information when copying its source. id-Software/... +1

What about the original source?

Keep one untouched copy in:

```
~/quake-dev/base-source/quake-rerelease-qc/
```

Whenever you start a new project, copy it:

```
cp -a ~/quake-dev/base-source/quake-rerelease-qc \
      ~/quake-dev/projects/my-new-mod/source
```

This gives you:

- An untouched reference copy.
- Completely independent source code for every mod.
- Independent Git histories.
- No risk of accidentally sharing modifications between projects.

My recommendation: Do exactly what you suggested. A separate source folder inside each mod project is a sensible and straightforward workflow.

Later, if you develop several mods that share code, you can look into Git submodules or shared libraries. But don't introduce that complexity yet.

You said:

ok how to add a new enemy. just the basic workflow

ChatGPT said:

The basic workflow is:

Create the enemy's QuakeC logic → register its source file → connect its model and animations → compile → place it in a map → test it.

In the Quake rerelease source, enemies are implemented in QuakeC. The engine loads the compiled `progs.dat`, not the individual `.qc` files. id-Software/... +1

1. Create a new enemy source file

For example, suppose your enemy is called `monster_guard`.

Inside your project's source directory, create:

```
quakec/
└── monsters/
    ├── ...
    └── guard.qc
```

I'd keep custom enemies in their own files rather than putting all their code into `monsters.qc`.

2. Implement the enemy

Your new `.qc` file will contain several things:

- Model and sound precaching.
- Animation frame definitions.
- Idle and standing behaviour.
- Walking and chasing behaviour.
- Attack behaviour.
- Pain reaction.
- Death behaviour.
- Main spawn function.

The spawn function is particularly important. It initializes the enemy's properties, such as:

- Health
- Collision box
- Movement type
- Model
- AI behaviour functions
- Starting animation

For reference, look at the existing `monster_tarbaby` implementation in the original source. It is a useful example of a complete enemy spawn function. GitHub

3. Register the new source file

This is an important step.

Open:

```
quakec/progs.src
```

You'll find a list of all source files that the compiler should include.

Near the bottom, you'll see something like:

```
monsters/ogre.qc
monsters/fiend.qc
monsters/shambler.qc
monsters/knight.qc
...
monsters/enforcer.qc
monsters/shub.qc
```

Add your file:

```
monsters/guard.qc
```

Without this entry, FTEQCC won't compile your new enemy's code. GitHub

4. Add the enemy's assets

Your mod's game directory needs the enemy's model and sounds.

For example:

```
mymod/
├── progs.dat
├── progs/
```

```
│   └── guard.mdl
└── sound/
    └── guard/
        ├── attack.wav
        ├── pain.wav
        └── death.wav
```

Your QuakeC code references these paths when precaching the assets.

For example:

```
precache_model2("progs/guard.mdl");
precache_sound2("guard/attack.wav");
```

The paths must match the actual locations of the files.

5. Compile

Run FTEQCC again.

It will compile all the source files listed in `progs.src` into a new `progs.dat`.

Copy the resulting file into your mod's game directory.

6. Place the enemy in a map

This is how you actually get your enemy into the game world.

In Quake, map entities have a `classname` property.

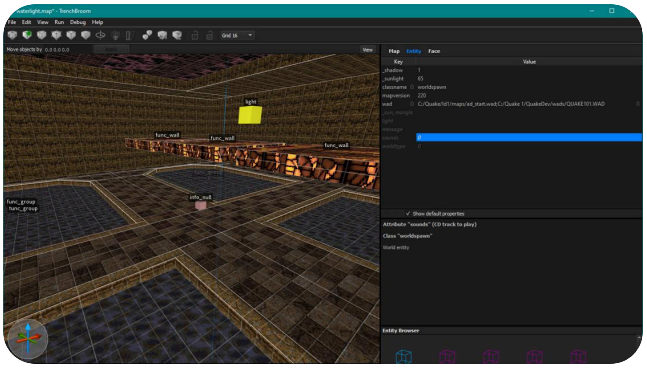
Your enemy's spawn function is:

```
void() monster_guard =
{
    // Initialization
};
```

The corresponding map entity must have:

```
"classname" "monster_guard"
```

You can place it using a Quake map editor, such as TrenchBroom.



7. Test

1. Launch your mod.
2. Load the test map containing your enemy.
3. Check that the model appears.
4. Check animations and movement.
5. Test attacks, damage and death.
6. Check the console for missing assets or runtime errors.

The whole workflow at a glance

1. **Create** `monsters/guard.qc`

   Write the enemy's QuakeC logic.
2. **Register it in** `progs.src`

   Tell the compiler about the new file.
3. **Add assets**

   Put models and sounds in the mod directory.
4. **Compile**

   Generate a new `progs.dat`.

5 **Place it in a map**

Add an entity with classname `monster_guard`.

6 **Test**

Run the map and verify its behaviour.

One important distinction: `monsters.qc` contains shared monster infrastructure, while individual monster files contain the actual implementations. You generally want to follow the latter approach for your own enemies.

For your first experiment, I'd actually recommend copying an existing monster's `.qc` file, renaming its functions and spawn function, and getting that working as a separate enemy. It's a much easier starting point than implementing everything from scratch.

You said:

this is the hellnight.qc
give me a detailly commented version of it.

ChatGPT said:

Absolutely. I can annotate it line by line, explaining both what the code does and why it is written that way.

However, I don't see the `hellnight.qc` file attached to your message.

Please upload `hellnight.qc` here.

I'll return a fully commented version that:

- Explains every function and important variable.
- Explains the QuakeC syntax and engine-specific functions.
- Describes the enemy's AI, movement, attacks and animation logic.
- Explains how the different functions call each other.
- Preserves the original code and behaviour exactly.

I'll write the comments with someone learning QuakeC modding in mind, rather than assuming you already know the engine internals.

You said:

Pasted text(8).txt
Document

Worked for 2m 22s

ChatGPT said:

Completed **Annotated Hell Knight source**

Download hellknight_commented.qc ↗

I've added detailed comments covering:

- Model and animation frame declarations.
- QuakeC function syntax and frame sequencing.
- Projectile creation and aiming.
- Enemy AI, chasing and charging.
- Melee and ranged attacks.
- Pain reactions, death and gibbing.
- Enemy initialization and spawn configuration.

Important: I preserved the original executable code. I verified that removing comments from the annotated version leaves the original code and model directives unchanged.

I haven't run it through FTEQCC, but no functional code changes were made. Keep your original file as a clean reference.

You said:

how intelligent can a quake monster get?

Worked for 19s

ChatGPT said:

Surprisingly intelligent. You can make a Quake monster considerably more sophisticated than the original monsters. You're not limited to the classic behaviour of walking towards the player, getting close and attacking.

QuakeC is a programming language, not a fixed monster-behaviour scripting system. You can implement your own decision-making, memory, perception, tactical behaviour and coordination.

The important distinction is between **what QuakeC can implement** and **what the engine makes convenient to implement**.

1. What levels of intelligence are possible?

Here is a rough progression:

Level	Behaviour
1. Basic	Sees player, chases and attacks.
2. Reactive	Dodges, retreats, strafes and chooses between attacks.
3. Tactical	Uses cover, maintains distance, flanks and repositions.
4. Perceptive	Hears sounds, investigates noises, remembers last known player position.
5. Strategic	Uses different tactics depending on health, ammunition, distance and surroundings.
6. Coordinated	Communicates with other monsters, surrounds players, calls for reinforcements.
7. Adaptive	Remembers previous encounters and changes its tactics based on player behaviour.

These are not engine-defined tiers. They're examples of increasingly sophisticated systems you could implement.

2. What does the Quake rerelease source already give you?

This is where things get interesting. The source you downloaded already contains much of the infrastructure you need.

For example, look at `quakec/ai.qc`.

Existing functionality	What it allows
`visible()`	Check whether another entity can be seen.
`infront()`	Determine whether a target is in front of the monster.

`range()`	Categorize target distance.
`FindTarget()`	Find potential enemies.
`ai_run()`	Handle enemy pursuit and attack decisions.
`ai_run_slide()`	Strafe sideways while fighting.
`walkpathtogoal()`	Use the rerelease engine's navigation system.

GitHub +2

There are also some useful additions in the rerelease:

- `self.oldenemy` lets monsters remember a previous target.
- `self.search_time` tracks how long they retain knowledge of an enemy.
- `self.attack_state` supports different combat states.
- `self.combat_style` distinguishes ranged, melee and mixed combat.
- `self.allowPathFind` controls whether advanced pathfinding is used.

GitHub

Interestingly, the rerelease source already contains an advanced movement implementation called `ai_pathtogoal()`. It supports using the engine's bot navigation system to move monsters towards their enemies. It's conditional and disabled by default in the original `id1` configuration. GitHub

3. The biggest limitation: navigation

This is probably the biggest technical challenge when making genuinely intelligent monsters.

Consider a monster that wants to flank the player.

It needs to:

1. Understand where the player is.
2. Find a suitable position to approach from.
3. Find a route to that position.
4. Navigate around walls and obstacles.
5. Reconsider its plan if the player moves.

The rerelease has `walkpathtogoal()` for navigation using available engine navigation nodes.

However, there is an important compatibility issue for your project:

- The rerelease implements `walkpathtogoal()`.
- Ironwail currently exposes the function but returns zero and does not move the entity.

QuakeBook +1

So you shouldn't build your entire monster AI around that function if Ironwail compatibility is important.

You can implement your own waypoint-based navigation in QuakeC, though that involves additional work.

4. What about hearing and memory?

There is an important distinction here.

Quake already has mechanisms for monsters to become aware of hostile activity. For example, the original targeting code uses hostile events and fake entities representing projectile impacts to alert monsters.

But `sound()` itself is a sound-playing function, not a general-purpose hearing system.

You could implement your own sensory system:

Hearing:

- Detect a player firing a weapon.
- Detect nearby explosions.
- Investigate the location of a stimulus.
- Remember where the sound originated.

Memory:

- Store the last known player position.
- Remember where the player was last seen.
- Search nearby rooms after losing sight.
- Eventually abandon the search.

The relevant building blocks are already available in QuakeC, including entity fields, vectors, distance calculations and collision traces. GitHub +2

5. What I'd consider achievable

Imagine creating a particularly intelligent Knight-type enemy.



Its behaviour could look something like this:

When idle

It patrols a designated area. Occasionally it looks around and reacts to suspicious activity.

When it detects the player

It doesn't immediately charge. It evaluates the distance and selects an appropriate tactic.

During combat

- At long range, it uses magic.
- At medium range, it approaches while dodging incoming projectiles.
- At close range, it switches to melee.
- When surrounded, it attempts to retreat.

- When low on health, it becomes more aggressive or defensive, depending on its personality.

When it loses sight of the player

It goes to the last known position and investigates. It might search nearby areas rather than immediately returning to its idle state.

When fighting alongside other monsters

It communicates its target to nearby allies. One monster engages the player while another attempts to flank.

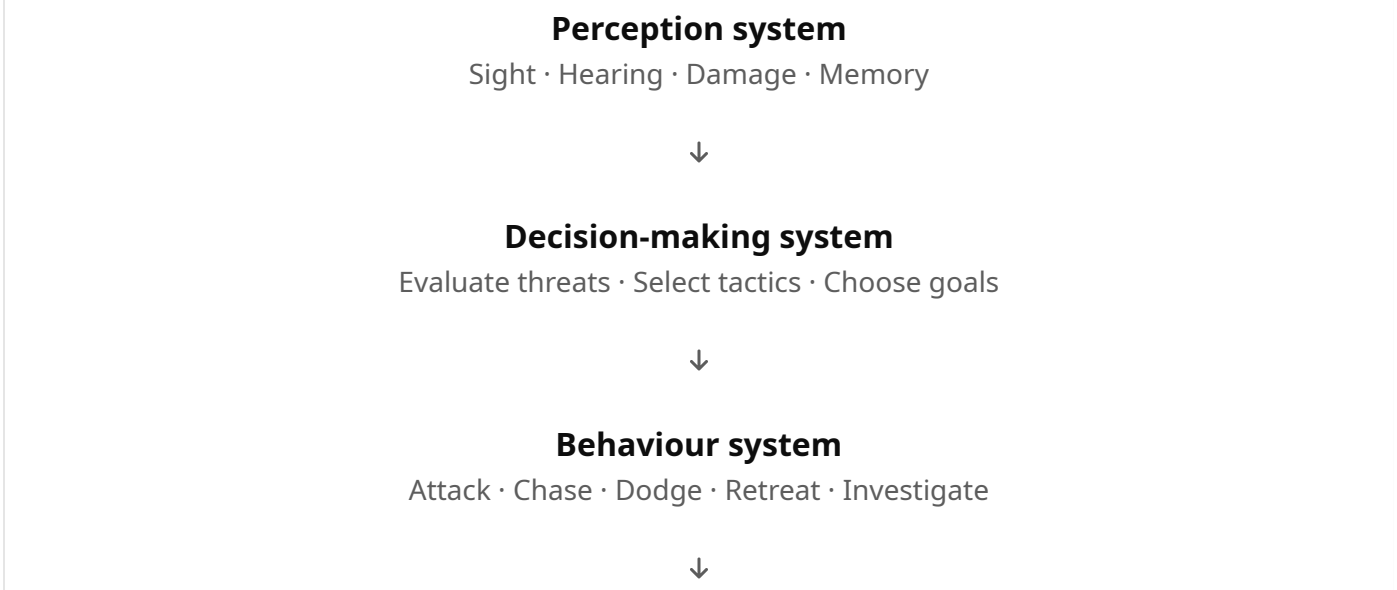
All of this is achievable with conventional QuakeC programming, although navigation and coordination require custom systems.

6. My recommendation: don't think of monsters as animation sequences

Looking at your `hellnight.qc`, you can see that its behaviour is primarily organized around animation frames and a handful of decisions.

For more sophisticated enemies, I'd gradually move towards a structure like this:

Monster AI architecture



Animation system
Play the appropriate model animations

The key is separating the monster's decision-making from its animation. Its AI decides *what* it wants to do; its animation system handles *how* it visually performs that action.

My take: QuakeC is capable of much more sophisticated enemy AI than Quake's original monsters demonstrate. Your main limitations will be navigation, engine compatibility and the amount of code you're willing to write—not the basic intelligence you can implement.

I'd aim for tactical, reactive monsters rather than trying to simulate human intelligence. That's where you can get some very impressive results without fighting the engine unnecessarily.

You said:

can a monster cast rays?

Worked for 10s

ChatGPT said:

Absolutely. QuakeC has a built-in function called `traceline()`, which performs a raycast.

And the good news: it's a standard Quake engine builtin, so it's suitable for your rerelease and Ironwail compatibility target. GitHub +1

1. Basic raycast

The function signature is:

```
traceline(start, end, nomonsters, ignore_entity);
```

For example:

```
local vector start;
local vector destination;

start = self.origin + '0 0 24';
```

```
destination = self.enemy.origin;

traceline(start, destination, FALSE, self);
```

This casts an invisible ray from the monster towards its enemy.

The parameters:

Parameter	Meaning
`start`	Ray origin
`end`	Ray destination
`nomonsters`	Whether other monsters should be ignored
`ignore_entity`	Entity to exclude from collision testing, usually `self`

2. How do you know what the ray hit?

QuakeC provides several global variables that are populated by `traceline()`:

Variable	Meaning
`trace_fraction`	How far the ray travelled (1.0 = unobstructed)
`trace_endpos`	Where the ray stopped
`trace_ent`	Entity hit by the ray
`trace_plane_normal`	Surface normal at the impact point
`trace_startsolid`	Whether the ray started inside solid geometry

These are declared in the rerelease's `defs.qc`. GitHub

For example:

```
if (trace_fraction < 1)
{
    // Something obstructed the ray.
}
```

Or:

```
if (trace_ent == self.enemy)
{
    // The ray hit our enemy.
}
```

3. Practical example: monster vision

Imagine a monster that can only see the player when there is no wall between them.

```
float() MonsterCanSeeEnemy =
{
    local vector start;
    local vector destination;

    // Start the ray around the monster's upper body.
    start = self.origin + '0 0 24';

    // Aim towards the enemy.
    destination = self.enemy.origin;

    // Cast the ray.
    // FALSE means other monsters can block the ray.
    // self is ignored during collision testing.
    traceline(start, destination, FALSE, self);

    // Did we hit the intended target?
    if (trace_ent == self.enemy)
        return TRUE;

    // No obstruction along the ray.
    if (trace_fraction == 1)
        return TRUE;

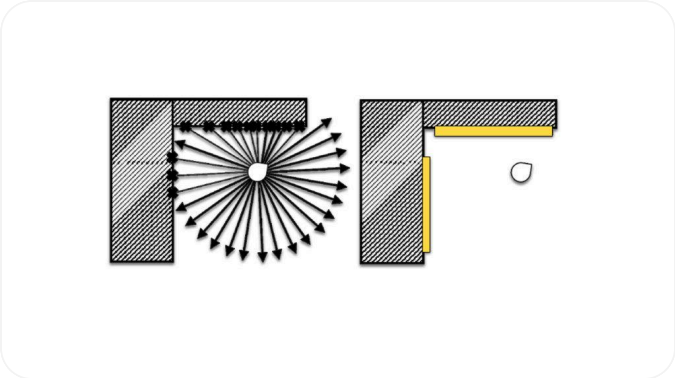
    // Something is blocking the monster's view.
    return FALSE;
};
```

This is a simplified visibility test, but illustrates the principle.

4. You can cast multiple rays!

This is where things get interesting.

Instead of casting one ray, you can cast several:



For example, a monster could cast:

- One ray towards the player's head.
- One ray towards the player's torso.
- One ray towards the player's feet.

This allows you to distinguish between situations where the player is completely hidden and situations where only part of the player is visible.

You could also use multiple rays for obstacle detection, checking whether a monster can move forward, and choosing an unobstructed direction for a projectile.

One important technical detail: `traceline()` performs a collision test, not a visible laser effect. Also, its results are stored in global trace variables, so save any results you need before performing another trace.

My recommendation: Get comfortable with `traceline()` early. It's one of the most useful tools for implementing sophisticated monster behaviour in QuakeC.

You said:

but it only casts for other entities or is also the pixel of a wall

texture possible?

Worked for 19s

ChatGPT said:

No. `traceline()` **cannot test individual pixels of a wall texture.** It tests the map's collision geometry, not the rendered image.

There are three different things to distinguish:

What you want to detect	`traceline()`
A wall or solid piece of world geometry	Yes
A solid entity, such as a door or monster	Yes
An individual pixel of a wall texture	No

For example, imagine a wall with a window painted onto its texture. Visually, the player can see through the window, but the underlying map geometry is still a solid wall.

A ray will hit that wall regardless of which part of the texture it appears to hit.

What about transparent textures?

This is particularly relevant in Quake.

Ironwail supports alpha-tested fence textures (usually texture names beginning with `{`). They can visually contain holes, but those holes are a rendering feature, not automatically holes in the collision geometry. QuakeBook



So a monster might see the player through a fence visually, while `traceline()` still reports that a wall blocks its view.

What can you do instead?

You have a few options:

1. **Actual geometric openings:** Build the wall with real gaps. Rays will pass through them naturally.
2. **Non-solid geometry:** Use non-solid entities for objects that should not block vision or projectiles.
3. **Custom visibility system:** Represent special transparent areas using helper entities or map-defined information and account for them in your AI.

There are also engine-specific extensions in some Quake engines that can retrieve surface texture information, but these aren't a portable way to sample individual pixels. The rerelease's standard QuakeC trace globals provide collision information, not texture pixel data. GitHub +1

For your monster AI: I'd distinguish between visual visibility and physical line of sight. Those don't always have to be the same thing. A monster might see you through a grate but still be unable to shoot you through it, for example.